42 ABU DHABI · CAPSTONE PROJECT · STAFF EVALUATION

Capstone Project: Ft_Transcendence

Salim Hamzaoui · shamzaou

Alisher Abdullaev · alabdull

Nasser Alzaabi · naalzaab

Nour Murat · nurmurat

FAST_PONG — a web platform for 3D Pong with tournaments, player statistics, 42 login, 2FA and GDPR tools

Evaluation date: ____ / ____ / 2026

CONTENTS

01

Introduction

Presenter: _____

Project Overview

Ft_transcendence is a web-based multiplayer gaming platform developed as the capstone project of the 42 Abu Dhabi curriculum. Its core is a modern 3D implementation of the classic Pong, surrounded by a complete user experience: secure accounts, 42 Intra login, email two-factor authentication, player profiles with statistics and match history, a friends list, a tournament system and GDPR tools.

The application is a Single Page Application: one server-rendered page, after which JavaScript swaps views without full reloads. The backend is Django (Python) with a PostgreSQL database; Gunicorn serves the site over HTTPS, and the whole stack runs in Docker Compose. The 3D game is rendered with Three.js (WebGL) and includes a computer-controlled opponent.

Backend

Django 4.2, Django REST Framework, SimpleJWT, Gunicorn (HTTPS on port 443)

Frontend

Vanilla JavaScript SPA, Bootstrap, Three.js for the 3D Pong scene

Data & Ops

PostgreSQL 13, Docker Compose, Git/GitHub feature-branch workflow

Project Objectives

The goal was to design, develop and deploy a fully functional and secure web application centred on a Pong game. The objectives set at the outset were:

Functional gaming platform

A fully operational website with 3D Pong (two players or versus AI) as its central attraction.

Secure authentication

Email/password registration, 42 Intra OAuth for students, optional email-based 2FA and JWT tokens.

Persistent user profiles

Display name, avatar, win/loss statistics, best score and complete match history.

Tournament mode

Create a tournament of 3–8 nicknames, play a round-robin of matches and determine a winner.

Application security & GDPR

Protection against SQL injection, XSS and CSRF; data export and account deletion.

Collaborative development

An Agile, feature-branch workflow with code reviews, Docker and a maintainable codebase.

Scope of the Project

The scope covers every essential aspect of a modern web application, from user management to gameplay and deployment:

User management

Registration, login, 42 OAuth, profile editing, avatar upload, friends list.

Pong (3D)

Local two-player mode on one keyboard and a single-player mode against the AI opponent.

Tournaments

Creation, nickname registration, automatic match generation, tiebreakers, winner.

Profiles & statistics

Games played, win rate, best score, recent matches, JSON export of all data.

Security & privacy

Hashed passwords, HTTPS, CSRF, 2FA + JWT, GDPR data export / deletion.

Deployment

Two containers (web, db) orchestrated with Docker Compose; one command to run.

Bonus feature

A local Tic-Tac-Toe game whose results also appear in the match history.

Team process

Agile iterations, Git feature branches, pull requests and peer review.

02

Software Development Life Cycle

Presenter: _____

Chosen SDLC Model: Agile (iterative & incremental)

The project was built in short cycles, each delivering one working feature that was merged and tested before the next one started. This suited a four-person learning project with evolving requirements.

1 Iterative development

Work was broken into features — authentication, Pong core, tournaments, profiles — each built in its own short cycle.

2 Incremental delivery

A runnable, testable version existed after every merge; functionality grew with each iteration.

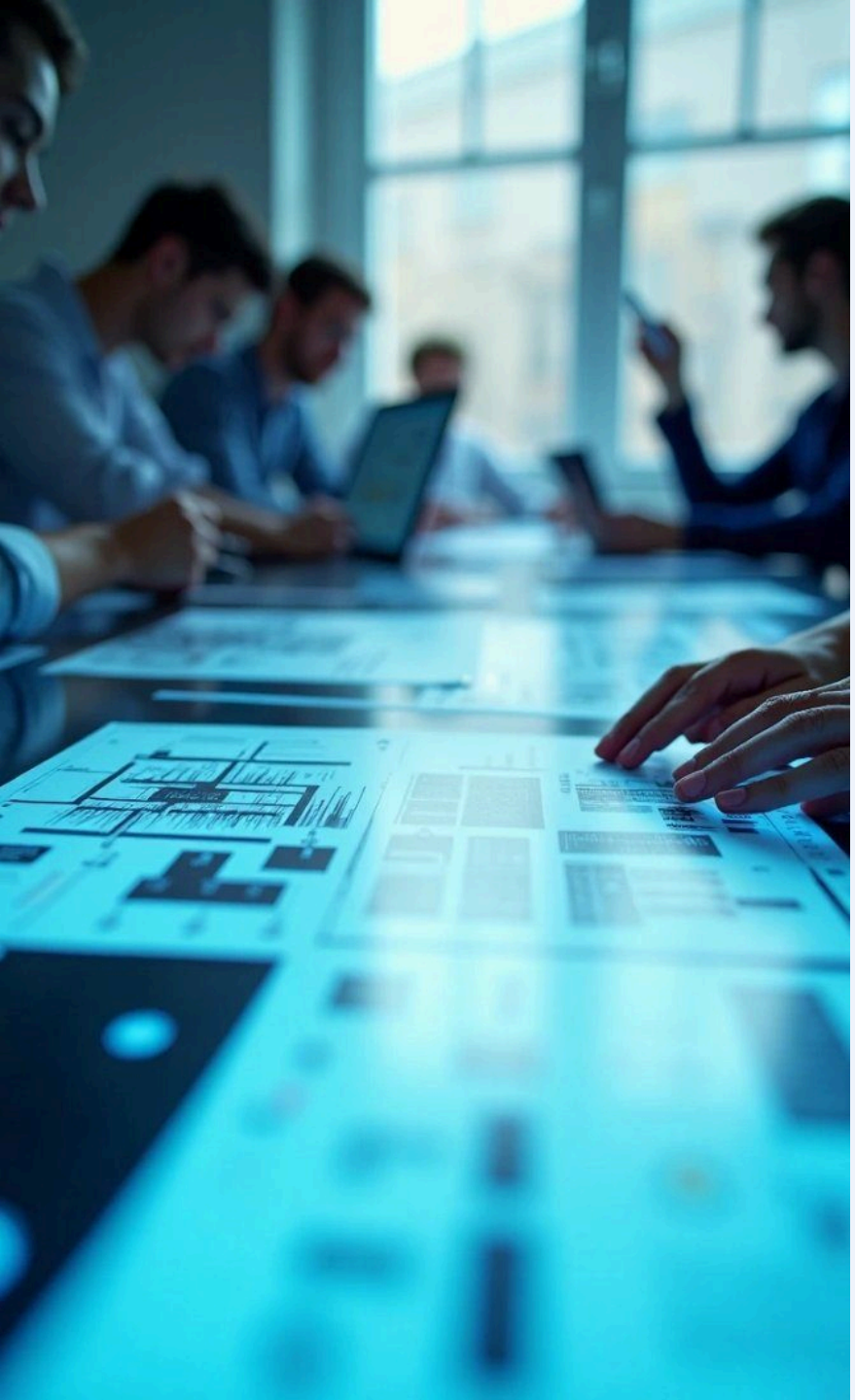
3 Flexibility

Requirements and the technical approach were refined as our understanding of the project grew.

4 Collaboration & feedback

Daily check-ins, pairing on complex features and peer review of every merge.

Evidence in the repository: 86 commits between 10 Feb and 2 Apr 2025, 15 pull requests merged from feature branches (db-connect, game-setup, tournaments, profile-page, secure-cookies, OAuth, user-settings, delete-account …).



Phases and Iterations

We did not use formal time-boxed sprints, but every major feature went through the same cycle of phases:

1 Planning

Discuss the next feature (e.g. 2FA, tournament logic), clarify objectives and acceptance criteria.

2 Design

Sketch data models, API endpoints and simple wireframes before coding.

3 Implementation

Develop the feature in a dedicated Git branch to avoid conflicts with the main codebase.

4 Developer testing

Unit tests for backend logic and functional checks by the developer.

5 Integration & review

Peer code review, then merge of the feature branch into master.

6 System testing

Test the feature inside the whole application to catch regressions.

Team Collaboration and Version Control

Clear processes and modern tools kept four developers working in parallel without stepping on each other.

Version control

Git with GitHub as the central repository — every change tracked and reversible.

Branching strategy

Feature-based branches (OAuth, tournaments, profile-page …) isolated work in progress from master.

Code reviews

Every branch was reviewed through a pull request before merging — 15 PRs in total.

Communication

A WhatsApp group for quick coordination and regular in-person meetings on campus.

Project Timeline (Gantt Chart)

The Gantt chart planned and tracked the project over time: task durations, dependencies and milestones.

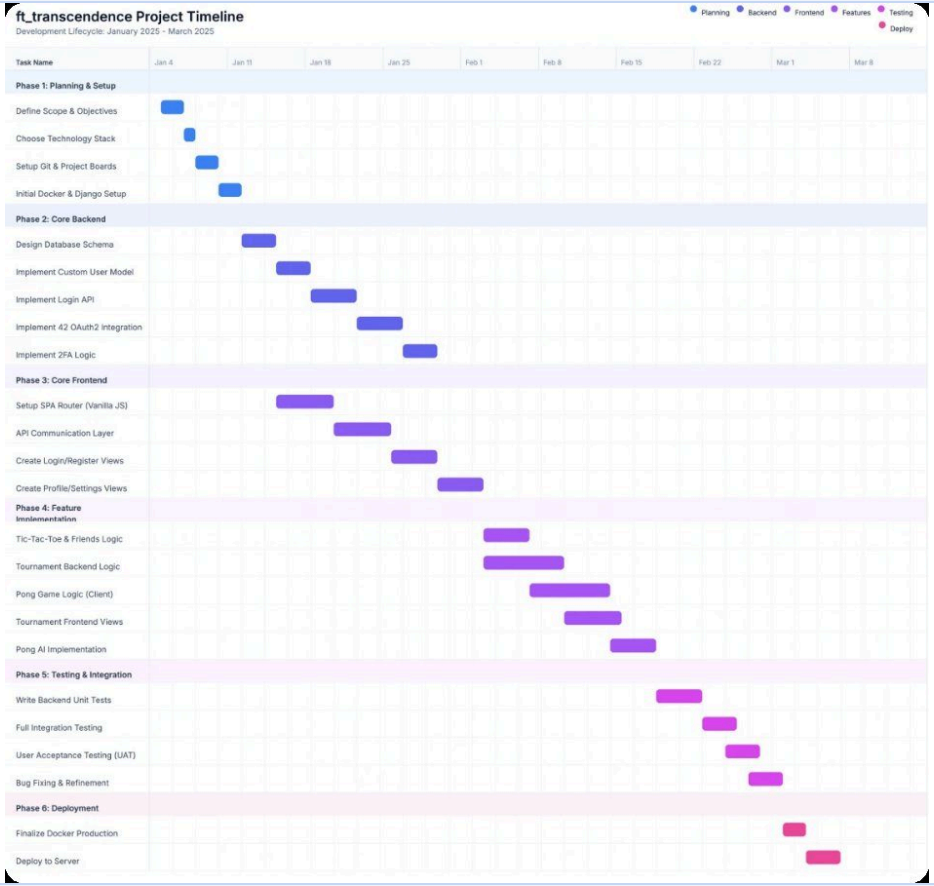


Figure 1: Project Gantt chart — planning & setup, core frontend, core backend, feature implementation, testing & integration, deployment.

03

Selected Modules

Presenter: _____

Selected Modules

Category	Module	Type	How it is implemented
Web	Use a framework as backend	**Major**	Django 4.2 + Django REST Framework
Web	Use a front-end framework or toolkit	**Minor**	Bootstrap 4.5 + custom CSS
Web	Use a database for the backend	**Minor**	PostgreSQL 13 via the Django ORM
User Management	Standard user management, authentication, users across tournaments	**Major**	Register / login, profiles, avatars, friends, stats; tournaments of nicknames
User Management	Implementing a remote authentication	**Major**	42 Intra OAuth 2.0 (authorize → callback → token exchange → JWT)
AI-Algo	Introduce an AI opponent	**Major**	PongAI: samples the ball once per second, predicts the intercept, tunable accuracy — no A*
AI-Algo	User and game stats dashboards	**Minor**	Profile cards, win-rate chart, match history, tournament scoreboard, JSON export
Cybersecurity	GDPR compliance: anonymization, local data management, account deletion	**Minor**	Data export, account deletion, inactive-account cleanup
Cybersecurity	Two-Factor Authentication (2FA) and JWT	**Major**	Email one-time code on login; SimpleJWT access / refresh tokens
Graphics	Use of advanced 3D techniques	**Major**	Three.js: perspective camera, lights, Phong materials, textured spinning ball
Accessibility	Support on all devices	**Minor**	Responsive layout, media queries, hamburger menu
Accessibility	Expanding browser compatibility	**Minor**	Standard ES modules / WebGL; Chrome and Firefox
Accessibility	Server-Side Rendering (SSR) integration	**Minor**	Django renders the initial page (CSRF token, static manifest); the SPA takes over

6 Major + 7 Minor (× 0.5) = 9.5 major-equivalents — 7 are required for 100 %. Not selected: another game, microservices, multiple languages (Tic-Tac-Toe is a bonus feature only).

04

Design

Presenter: _____

System Architecture

The application is a monolith with a clear separation between frontend and backend, running in containers:

- The browser loads one server-rendered page and then runs the SPA.
- Gunicorn (3 workers) terminates HTTPS on port 443 and runs the Django app; WhiteNoise serves static files.
- Django exposes the REST API and talks to PostgreSQL only through the ORM.
- External services: the 42 API for OAuth and Gmail SMTP for 2FA codes.
- Docker Compose defines the two services, the network and the database volume.

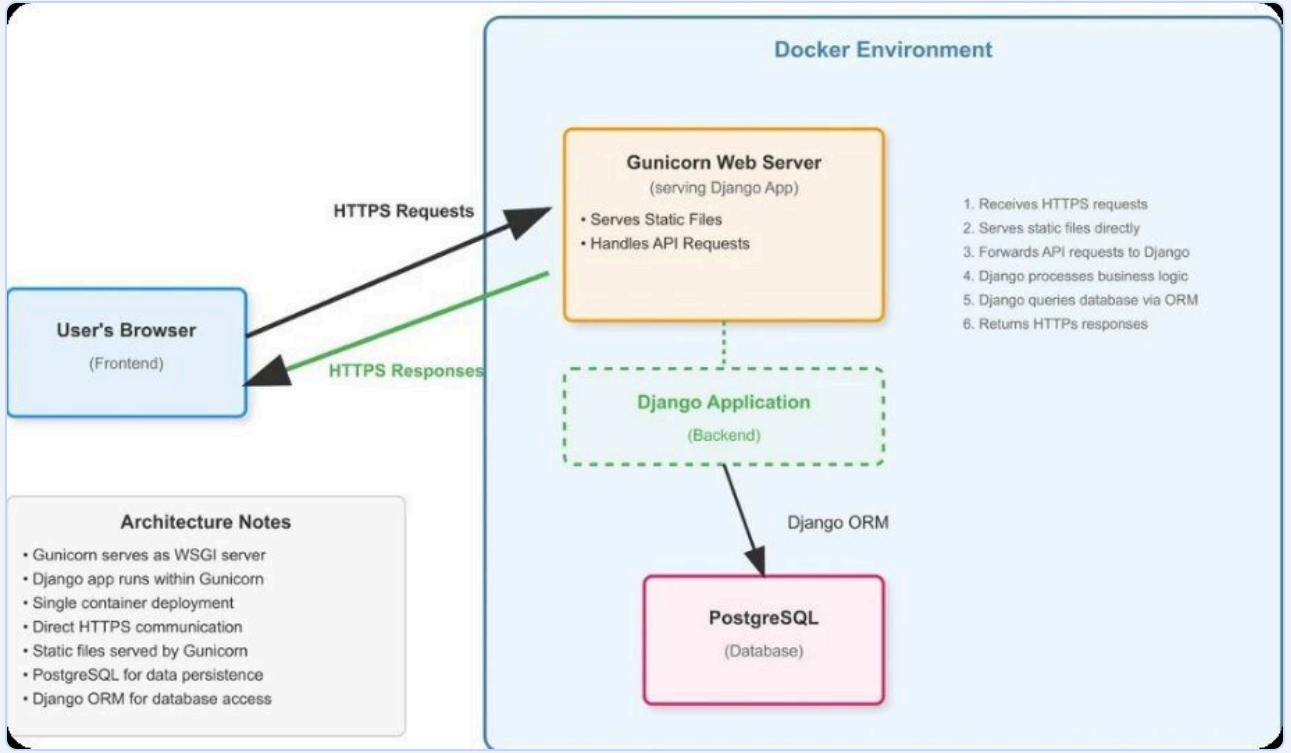
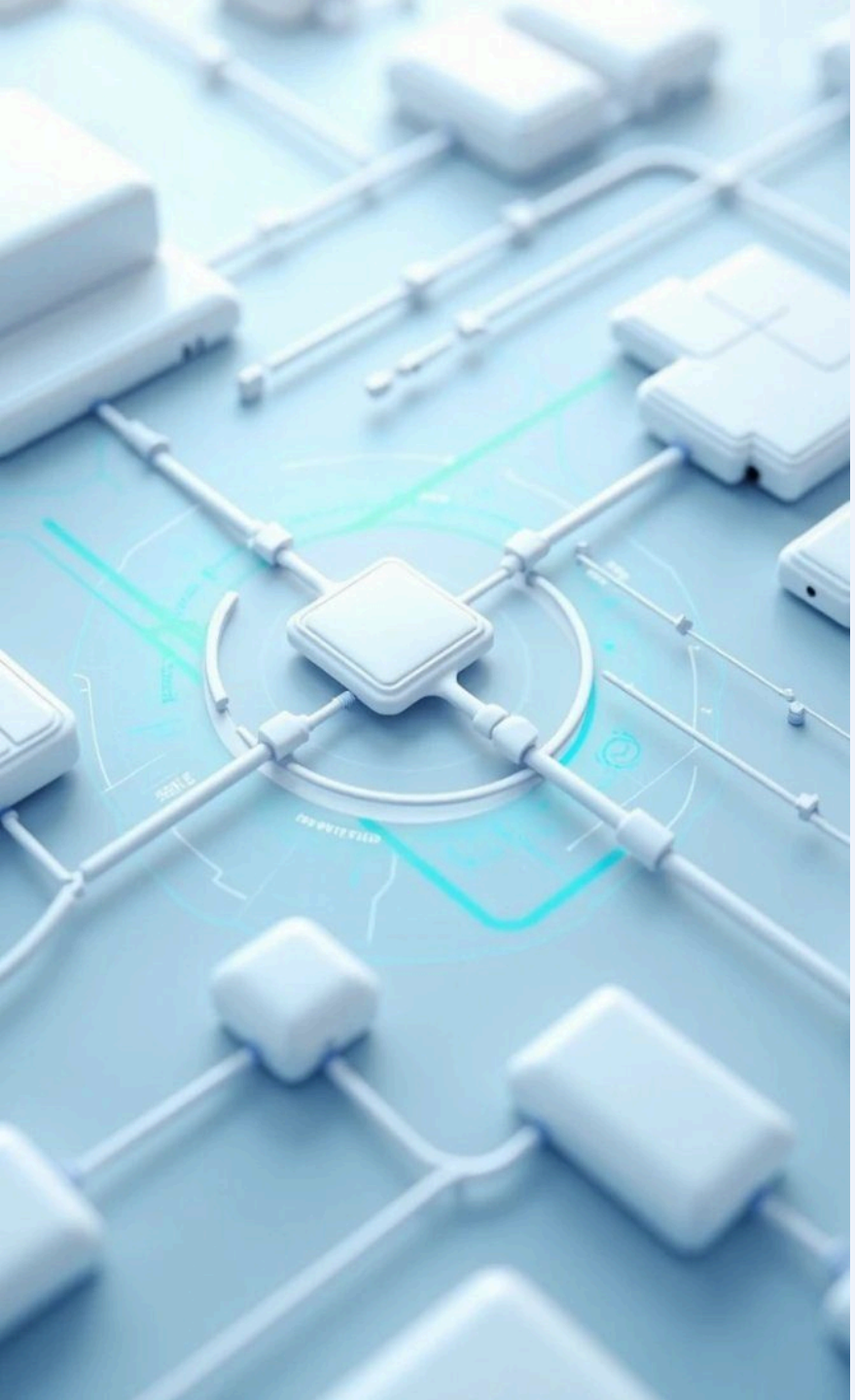


Figure 2: System architecture diagram



Database and API Design

The schema is defined with Django models, grouped into three apps; the frontend consumes a RESTful JSON API.

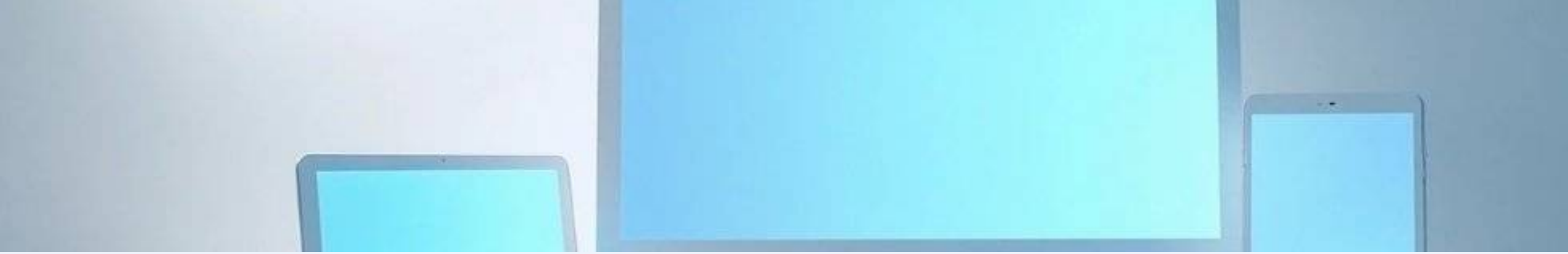
userapp

User (custom, email login, 2FA flag, avatar, friends M2M, last activity) and **MatchHistory** (game type, opponent, result, score).

tournaments

Tournament, **Player** (nickname per tournament) and **Match** (scores, winner, tiebreaker flag).

Endpoint	Purpose
POST /api/auth/register/ · login/ · logout/	Accounts and sessions (login starts 2FA)
POST /api/auth/verify-otp/	Check the emailed code, issue JWT
POST /api/auth/redirect_uri/ · get-token/	42 OAuth link and code exchange
GET/PUT /api/auth/profile/	Profile, stats, avatar, display name
/api/auth/save-match/ · match-history/	Record and list games
/api/auth/friends/… · users/	Friends list management
/api/auth/export-data/ · delete-account/	GDPR tools
/tournaments/api/tournaments/…	Create, add players, view, start / finish matches



UI / UX Design

The interface is built around clarity, simplicity and responsiveness: one consistent colour scheme and layout, immediate feedback on actions, and an app-like SPA experience.

SPA experience

No full-page reloads: the client-side router swaps views and keeps browser history working.

Responsive

Flexbox / grid layouts and media queries adapt the pages from desktop monitors to phones.

Feedback

Buttons react on hover, loading states are shown during API calls, results and errors are displayed clearly.

Wireframes & flowcharts

Screens and the gameplay / tournament flows were sketched before implementation.

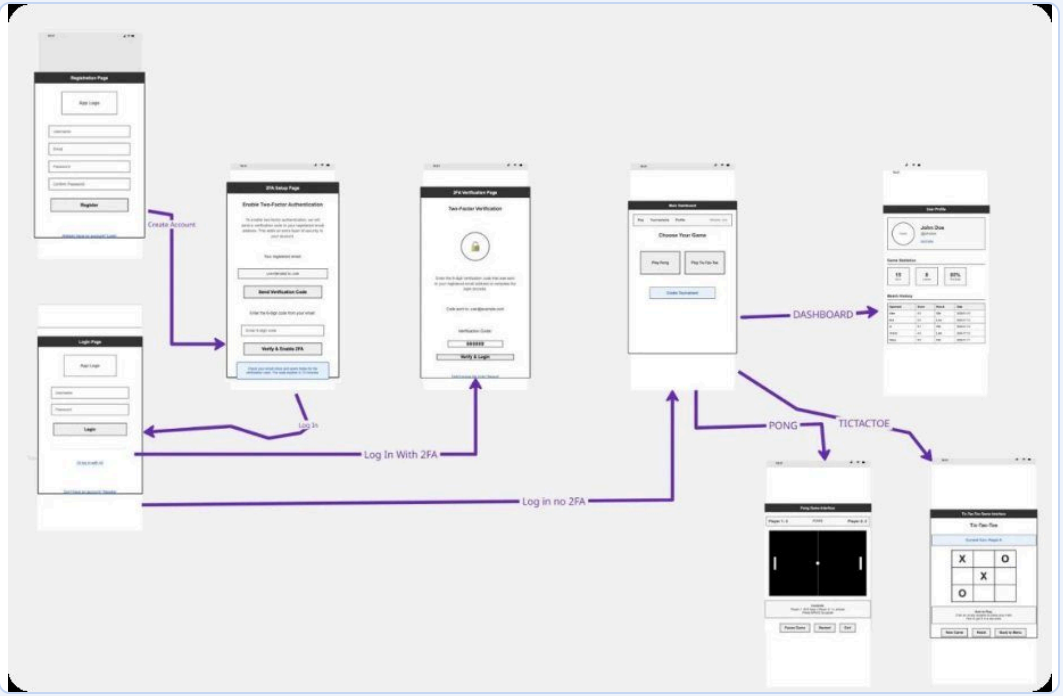
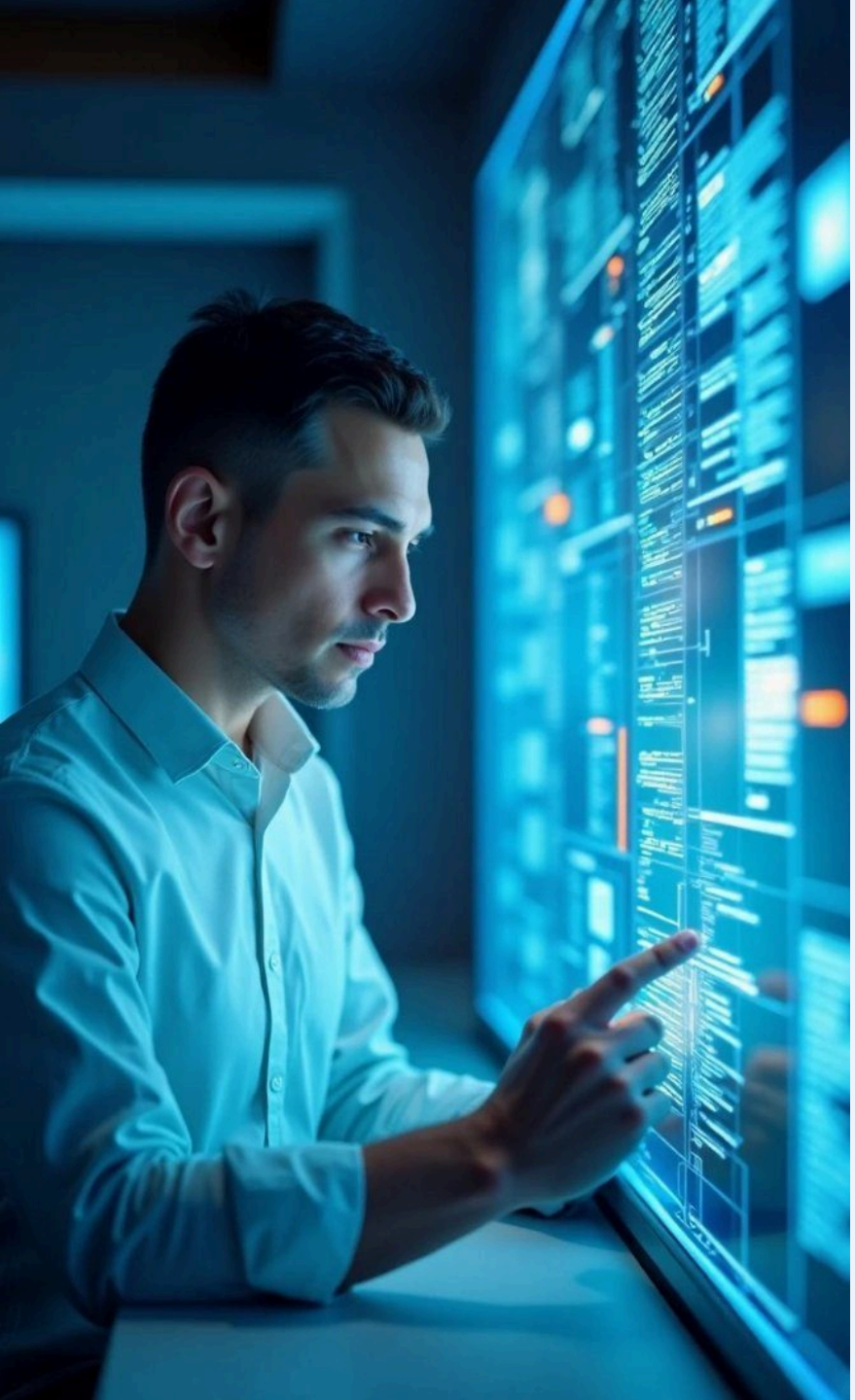


Figure 3: Wireframes of the key screens

05

Implementation & Features

Presenter: _____



Technology Stack

Chosen for robustness, simplicity and the learning objectives of the curriculum.

Backend

Python 3.11, Django 4.2, Django REST Framework, SimpleJWT, python-decouple for configuration

Server

Gunicorn with TLS on port 443 (self-signed certificate), WhiteNoise for hashed static files

Database

PostgreSQL 13 in its own container, accessed through the Django ORM and migrations

Frontend

Vanilla JavaScript SPA, HTML5, CSS3, Bootstrap 4.5

3D graphics

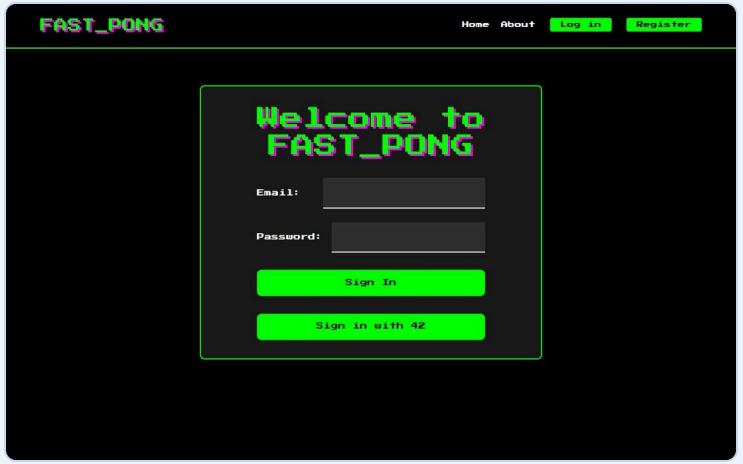
Three.js r128 (WebGL) for the Pong scene; HTML5 canvas for procedural textures

DevOps

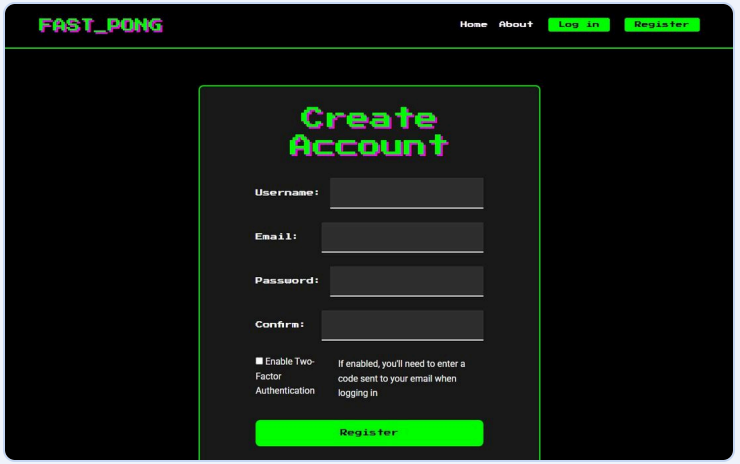
Docker Compose, Makefile targets, Git / GitHub with pull requests

Features — Authentication and 2FA

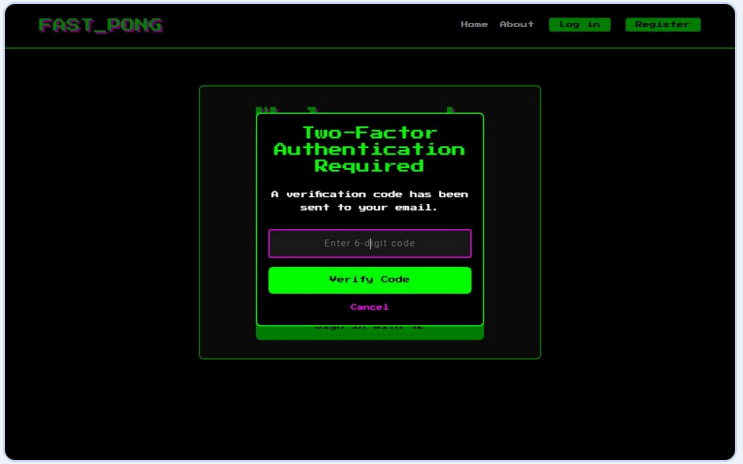
Email/password registration with a strong-password policy, 42 Intra login, and an emailed one-time code when 2FA is enabled.



Login page: email / password or “Sign in with 42”



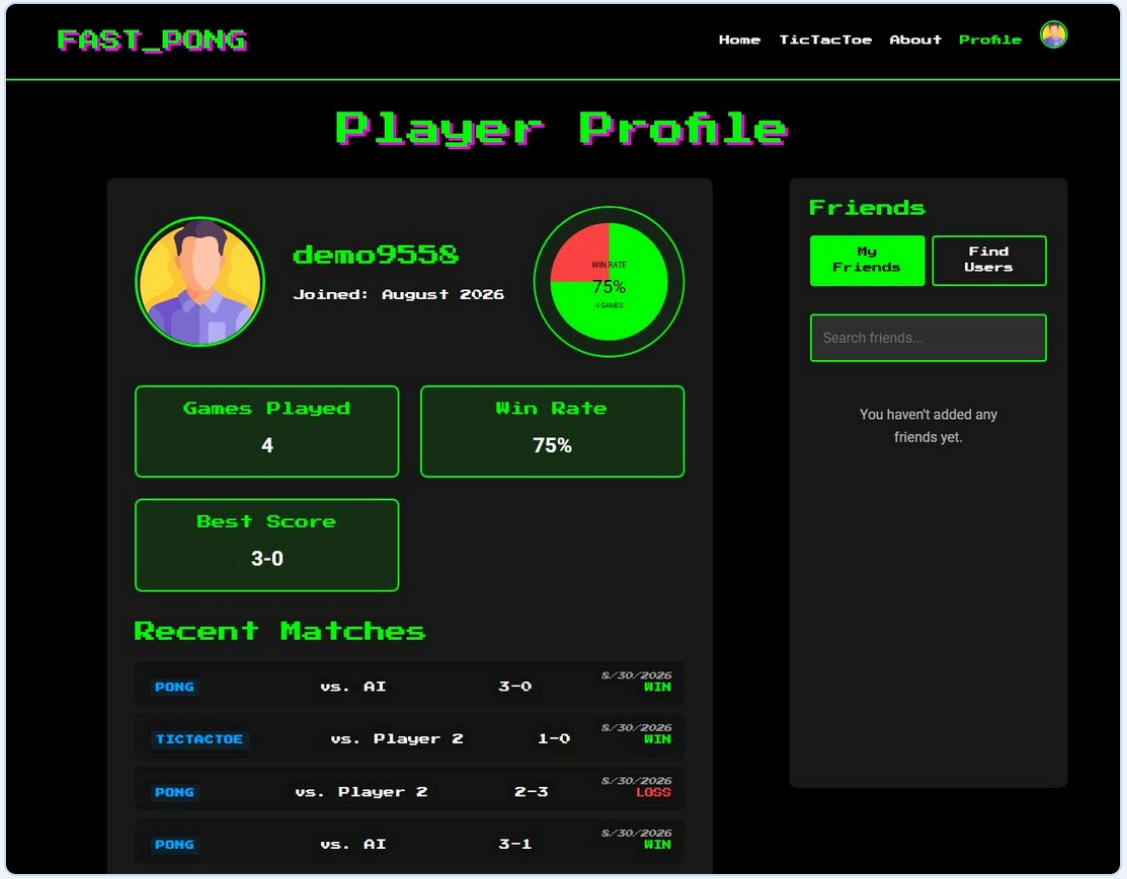
Registration with optional two-factor authentication



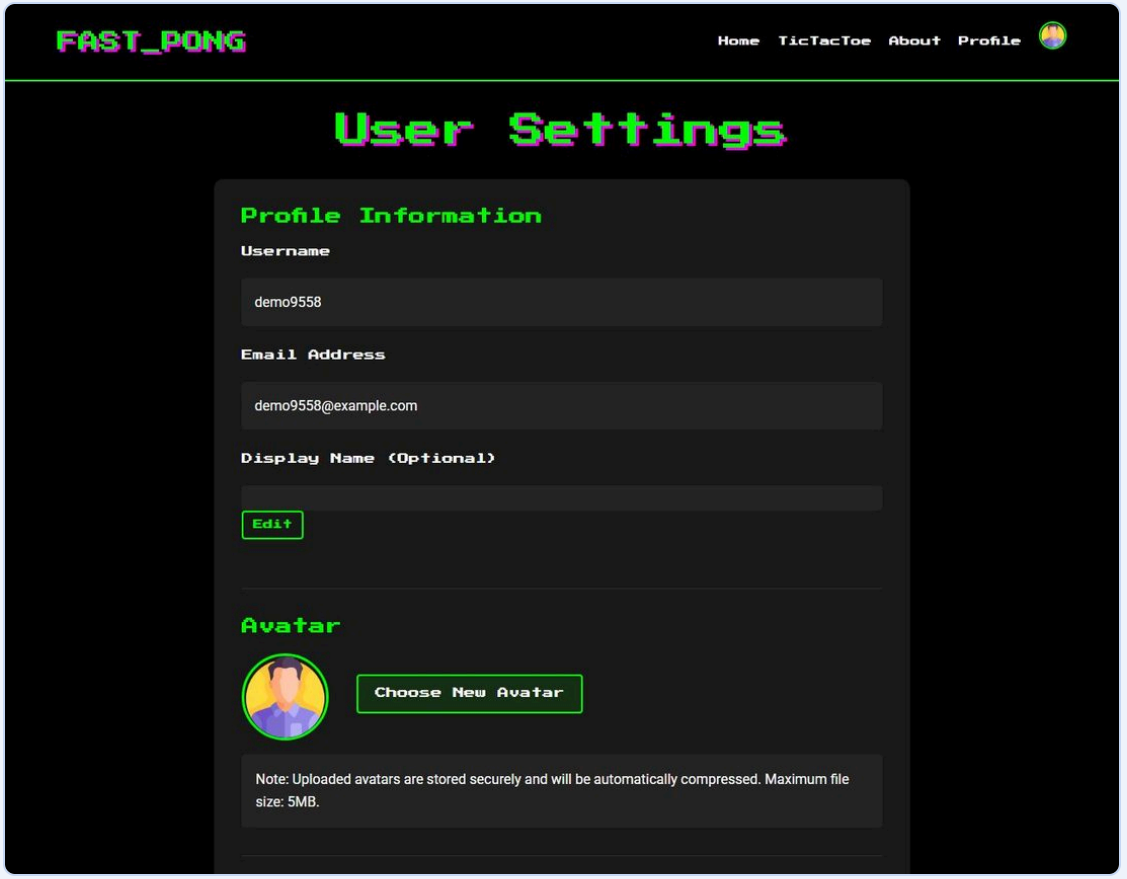
Second factor: 6-digit code sent by email, valid 10 minutes

Features — Player Profile and Stats Dashboard

Every finished game is recorded; the profile turns the history into a dashboard and the settings page holds the GDPR tools.



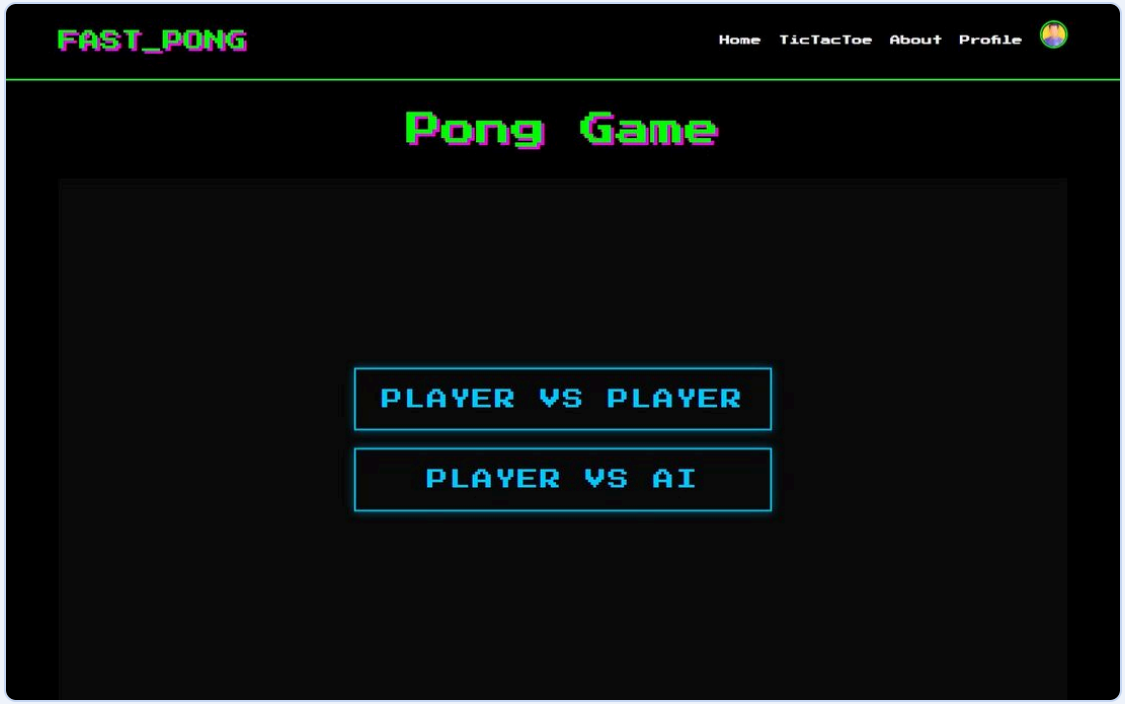
Games played, win rate chart, best score, recent matches, friends panel



Display name, avatar, Download my data, Delete account

Features — 3D Pong and the AI Opponent

Three.js scene with a perspective camera, lit table and a spinning textured ball. In “Player vs AI” the PongAI samples the ball once per second, predicts where it will cross the paddle line and moves with a tunable error margin — no path-finding algorithm.



Mode selection: Player vs Player (one keyboard) or Player vs AI

Game in progress against the AI — first to 3 points

Features — Tournaments (and the bonus Tic-Tac-Toe)

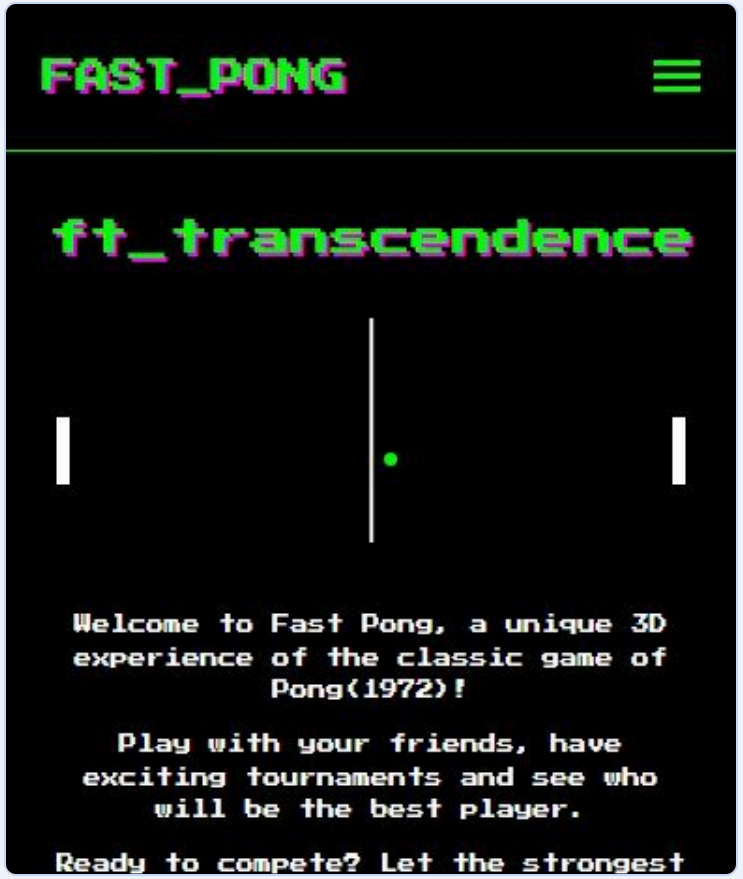
A logged-in user creates a tournament of 3–8 nicknames; every pair plays once, scores update the table, and tied leaders get automatic tiebreaker matches.

Round-robin schedule with “Start Match” per pairing and live scores

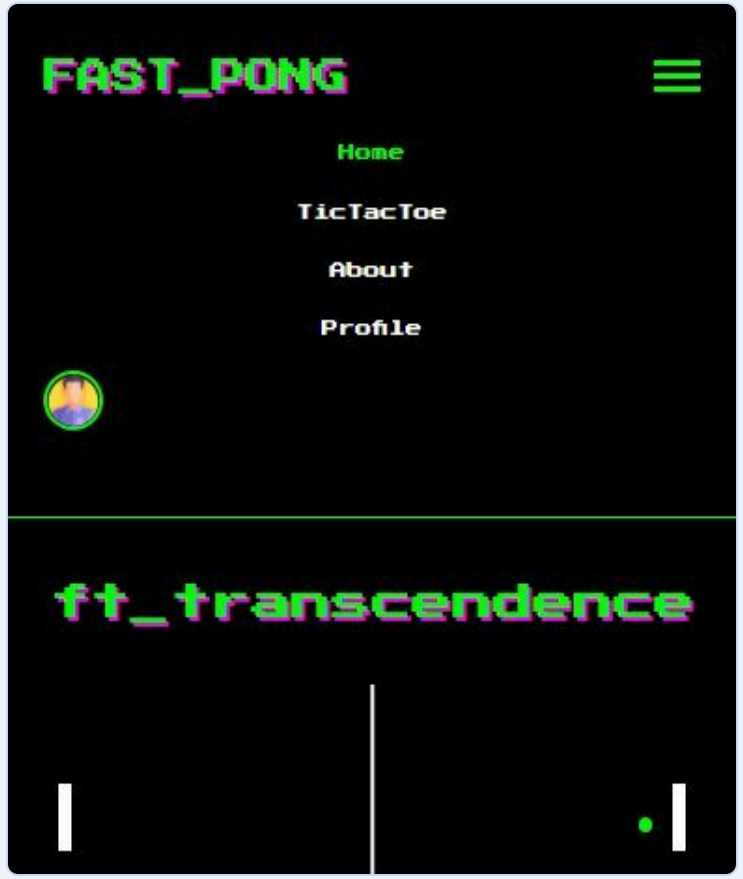
Bonus feature: local Tic-Tac-Toe, results saved to the match history

Features — Support on All Devices

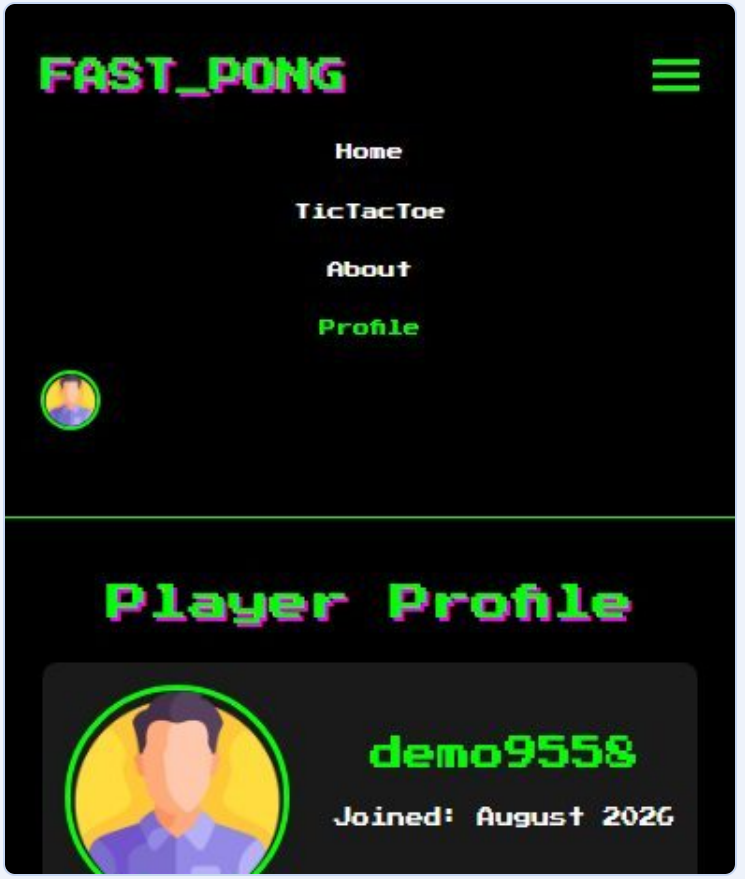
The same pages reflow for phones: hamburger navigation, stacked cards and a fluid game canvas.



Home on a 390 px phone



Hamburger menu



Profile stacked vertically

06

Security & GDPR

Presenter: _____

Cybersecurity Features

Security was a fundamental requirement, addressed in layers from the database to the browser.

Password storage

Django hashes and salts every password (PBKDF2); a validator enforces length, upper-case, digit and symbol.

2FA + JWT

Optional emailed one-time code as a second factor; SimpleJWT access (60 min) and refresh (7 days) tokens.

42 OAuth

Students log in through the 42 Intra; the server exchanges the code and never sees a password.

SQL injection

All database access goes through the ORM, which parameterises every query.

CSRF & XSS

Django’s CSRF token is required on every state-changing request; user data is inserted with textContent.

Transport

HTTPS everywhere — Gunicorn terminates TLS directly on port 443.

GDPR Compliance

Users own their data: they can take it with them or erase the account — and inactive accounts are cleaned up automatically.

Local data management

“Download my data” exports profile, statistics and full match history as JSON.

Account deletion

Hard-deletes the account and everything attached to it in one click, after confirmation.

Inactive accounts

A management command warns after 5 months of inactivity and deletes after 6 (last activity tracked by middleware).

The privacy policy on the About page describes the data collected, its use, retention and the user’s rights.

07

Testing & Evolution

Presenter: _____

Testing Strategy

Several levels of testing, each with its own focus, keep the application stable and secure.

Unit tests

Django test suite — 34 tests covering login and the whole 2FA flow, GDPR export / delete, the inactive-account command and tournament tiebreakers (`make test`).

Integration tests

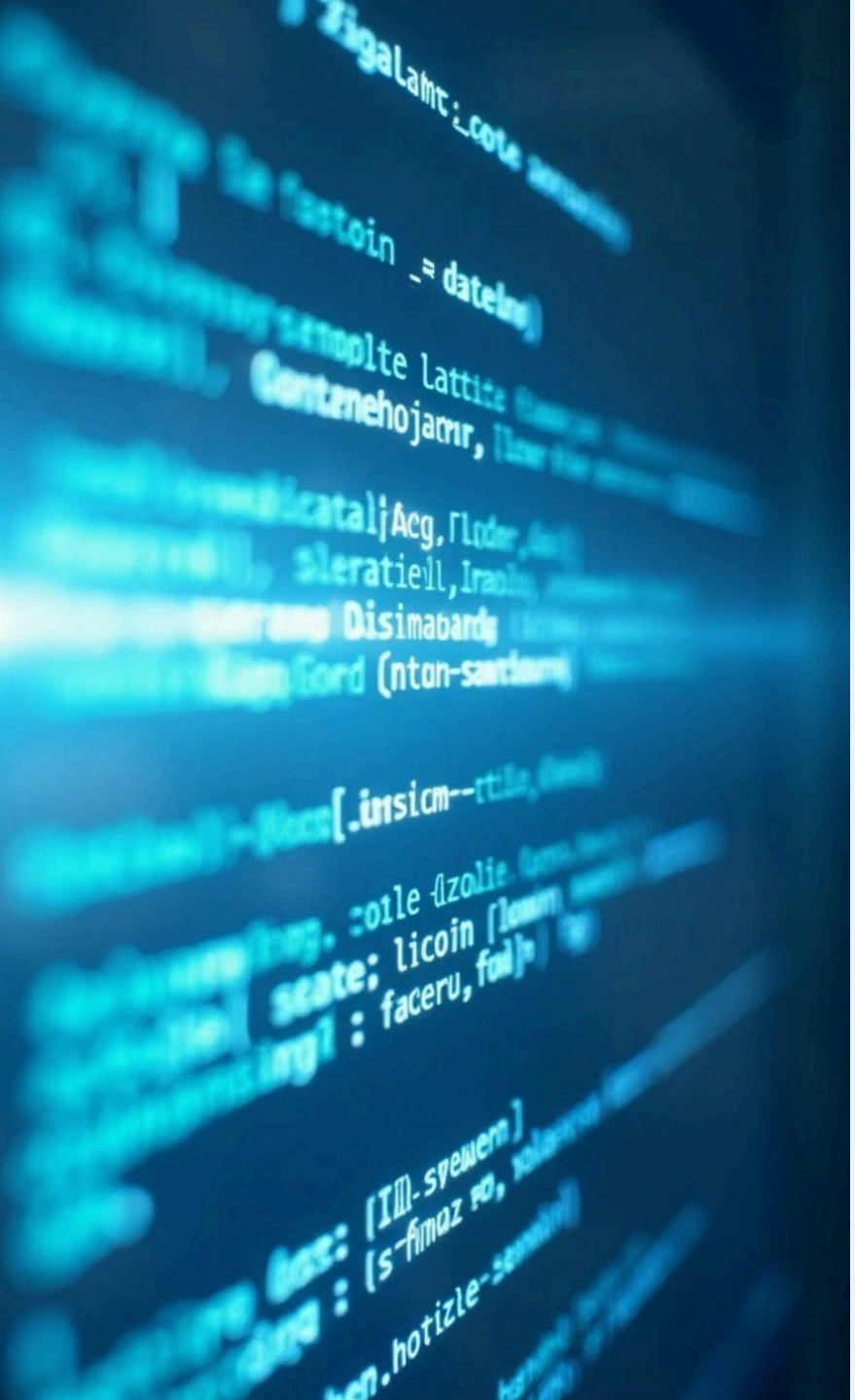
Scripted end-to-end API flow: register → login → profile → matches → friends → export → tournament → delete.

Browser walkthrough

Headless Chrome drives every page on desktop and phone, plays both games and checks for JavaScript errors.

Manual / acceptance

Team members continuously tested each other’s features from an end-user perspective.



Pre-Evaluation Audit (August 2026)

Before the evaluation the codebase was audited end-to-end. Two bugs reported by users were traced to their root causes and fixed with regression tests:

“The 2FA code is sometimes rejected”

The one-time code was kept in Django’s default in-memory cache, which is private to each process — and Gunicorn runs three workers. The verification request often reached a worker that had never seen the code. Fix: a database-backed cache shared by all workers.

“The 2FA e-mail is very slow”

The e-mail was sent synchronously inside the login request with no timeout, so the response waited for the whole SMTP round-trip (and failed with 500 on any error). Fix: send in a background thread with a 10 s timeout — login now answers in ~80 ms.

Also fixed

`make test` configuration, stale static files, the token-refresh URL in the SPA, a settings-page bug that saved a placeholder display name, the Pong ball getting stuck gliding along the top/bottom wall (wall bounce now clamps the ball and keeps a minimum angle), and the previous account's avatar remaining visible after switching users.

Result

A second sweep on request found and fixed 30 more bugs (silent JWT expiry after 60 min, secrets in logs, duplicate-registration errors, tournament persistence and repeated tiebreakers, Save Settings, 2FA toggle, Pong resize/touch/pause leaks, input validation). 34/34 tests pass, clean build verified, 0 JavaScript errors across the full browser walkthrough.

Challenges and Lessons Learned

Building a complete application from scratch provided technical and organisational hurdles — and lasting lessons.

Tournament logic

Generating fair schedules and resolving ties correctly required careful data modelling and state management.

State in vanilla JS

Without a framework, login state, routing and views had to be managed by hand with disciplined code structure.

Asynchronous flows

OAuth redirects, 2FA and API calls demanded a solid grasp of Promises and loading states.

Docker networking

Getting the web and database containers, volumes and environment variables to cooperate took iteration.

Lessons

A well-defined API, a mature framework, a disciplined Git workflow and security from day one all paid off.

Limitations and Future Enhancements

The current version meets every selected module; the following points are known limitations and the improvements we would make next.

Known limitations

Local multiplayer only (no online matchmaking); tournament API protected by CSRF but not by login; JWT stored in localStorage; the 42 OAuth flow has no *state* parameter; third-party assets load from CDNs; the 2FA mailbox needs a valid Gmail app password.

Next steps

1. Real-time online multiplayer and live chat with WebSockets (Django Channels). 2. Unified authentication on JWT with protected tournament endpoints. 3. OAuth *state* parameter and rate limiting on login / 2FA. 4. AI difficulty levels driven by the live score. 5. Leaderboards, achievements and 2FA recovery codes.

08

Team & Conclusion

Presenter: _____

Contribution of Each Member

Areas of responsibility derived from the Git history (commits, pull requests and files touched). Team: adjust the wording before presenting.

Salim Hamzaoui

SPA architecture and routing, page templates and styling, integration of the Pong and Tic-Tac-Toe games, profile / friends / statistics UI, avatar upload, GDPR pages, HTTPS on port 443, OAuth redirect flow — 66 commits, most merges.

Nasser Alzaabi

JWT authentication and the e-mail 2FA flow, 42 OAuth token exchange, Docker / port 443 configuration, match-result API and statistics for both games.

Alisher Abdullaev

Profile page and user-settings API, display name and e-mail editing, account deletion, user model migrations, Gantt planning.

Nour Murat

Tournaments app (models, views, URLs, templates and styles), round-robin scheduling and tiebreaker system, Pong integration for tournament matches.

Conclusion

Ft_transcendence delivered a feature-rich, secure and engaging web gaming platform that satisfies all the selected modules — 6 Major and 7 Minor, 9.5 major-equivalents against the 7 required.

Using Django, a vanilla-JavaScript SPA, Three.js and PostgreSQL inside Docker, the team gained hands-on experience in full-stack development, deployment and application security. The Agile, feature-branch workflow kept four developers productive and the codebase reviewable.

The modular structure is a solid base for the next evolution — real-time online play, richer AI and social features — while the pre-evaluation audit leaves the project with a green test suite and documented, root-caused fixes.

Thank You